

# Programmation Orientée Objet 2 POO2



Université Saad Dahlab  
de Blida



## CHAPITRE 1 Gestion des Exceptions

Mme BOUTOUMI  
2 Année Ingénieur  
2024- 2025

# 1. Introduction (1)

La qualité du logiciel recouvre :

- Un logiciel est **valide** s'il est capable de réaliser exactement les tâches définies par sa spécification.
- Un logiciel est **fiable** s'il est capable d'assurer de manière continue le service attendu.
- Un logiciel est **extensible** s'il est capable de s'adapter aux changements de la spécification.
- Un logiciel est **robuste** s'il est capable de fonctionner même dans des conditions anormales.

# 1. Introduction (2)

**Exemple:** Un programme qui calcule et affiche la factoriel d'un nombre entier positif entrés par l'utilisateur est valide s'il affiche le bon résultat quelques soient le nombre entré. Il est robuste si, en plus, il traite les cas ou le nombre introduit est négatif, réel ou non numérique en affichant un message d'erreur.

```
public class Test {  
    public static void main(String[] args) {  
        int n,fact=1; n=Integer.parseInt (args [0]);  
        if (n<0) System.out.println("n'est pas défini pour les nombres négatif");  
        else {  
            for(int i=2;i<=n;i++) fact*=i;  
            System.out.println(fact);  
        }  
    }  
}
```

# 1. Introduction (3)

```
public class Test {  
    public static void main(String[] args) {  
        int n,fact=1; n=Integer.parseInt (args [0]);  
        if (n<0) System.out.println("n'est pas défini pour es nombres négatif");  
        else {  
            for(int i=2;i<=n;i++) fact*=i;  
            System.out.println(fact);  
        }  
    }  
}
```

Exception in thread "main" java.lang.NumberFormatException: For input string: "3.2"  
at java.lang.NumberFormatException.forInputString(Unknown Source)  
at java.lang.Integer.parseInt(Unknown Source)  
at java.lang.Integer.parseInt(Unknown Source)  
at Test2.main(Test2.java:5)

- Même si un programme est correct, son exécution peut être interrompue brutalement à cause d'évènements exceptionnels (types de données incorrects, fin prématurée de tableaux ou de fichiers..etc). Ces évènements sont appelés Exceptions.



# 1. Introduction (4)

**Une exception est un évènement qui arrive durant l'exécution d'un programme qui conduit le plus souvent à l'arrêt de celui-ci.**

# 1. Introduction (5)

- Le concepteur d'un programme doit essayer dans un premier temps de prévoir les erreurs possibles soit:
  - En ajoutant des séquences de tests pour les empêcher,
  - En mettant en place un système pour signaler un fonctionnement anormal du programme en retournant des valeurs spécifiques
- Mais aucune de ces solutions n'est satisfaisante car:
  - il est très difficile de dénombrer toutes les erreurs possibles sans en omettre aucune,
  - l'écriture du code devient fastidieuse et complexe et sa maintenance difficile à cause de son illisibilité (noyé dans des imbrications de tests)

## 2. Exceptions

- De nombreux langages de programmation de haut niveau tel que **Java** possèdent un mécanisme permettant de gérer (détecter et traiter) les exceptions qui peuvent intervenir lors de l'exécution d'un programme.
- Avec le mécanisme de gestion des exceptions, un programme peut gérer des situations exceptionnelles sans que le code soit encombré partout avec des blocs de code qui ne sont presque jamais exécutés.
- La gestion des exceptions s'appelle aussi la capture d'exception.

### 3. Exemple d'exception

```
public class TestException {  
  
    public static void main(String[] args) {  
  
        int i = Integer.parseInt (args [0]);  
        int j = Integer.parseInt (args [1]);  
        System.out.println("résultat = " + (i / j));  
  
    }  
}
```

Si l'utilisateur saisit un zéro comme deuxième valeur

**Exception in thread "main" java.lang.ArithmeticException: / by zero**  
**at TestException.main(TestException.java:6)**

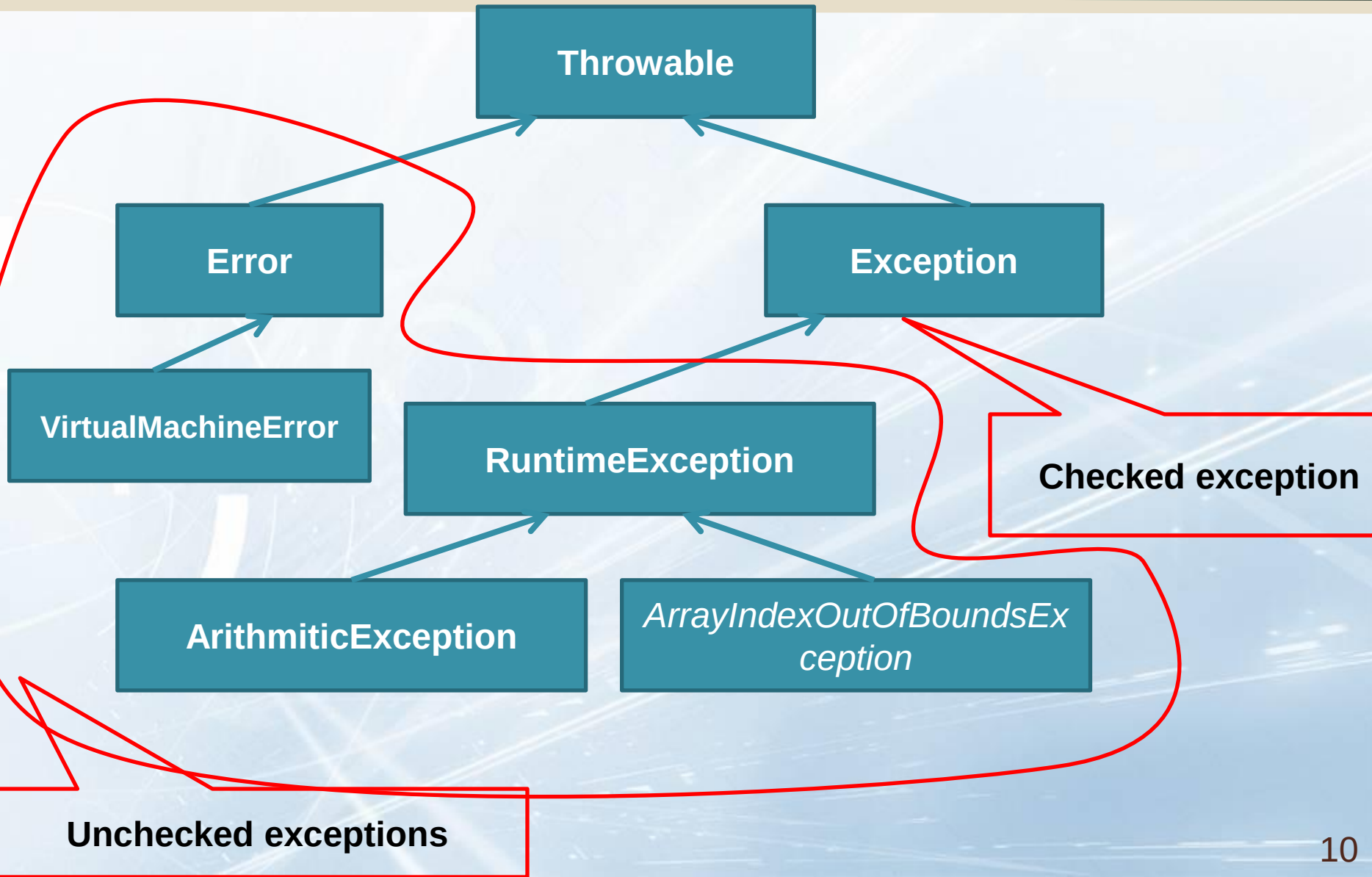


## 4. La hiérarchie des exceptions (1)

Il existe deux catégories d'exceptions :

1. **Unchecked exceptions** : regroupe les exceptions de type `Error` et `RuntimeException`;
  - ❖ **Error ou ses sous-classes** : ces exceptions concernent des problèmes liés à l'environnement. Elles héritent de la classe `Error` (exemple : `OutOfMemoryError`).
  - ❖ **RuntimeException** : ces exceptions concernent des erreurs de programmation qui peuvent survenir à de nombreux endroits dans le code (exemple : `NullPointerException`). Elles héritent de la classe `RuntimeException`
2. **Checked exception** : ces exceptions doivent être traitées ou propagées. Toutes les exceptions qui n'appartiennent pas aux catégories précédentes sont de ce type. Les checked exception peuvent être standard (définie dans JDK) ou bien personnalisée.

## 4. La hiérarchie des exceptions (2)



## 5. Traitement d'une exception

- Un programme java bien écrit qui contient une partie du code pouvant générer ( ou provoquer) une exception contrôlée doit essayer de la traiter (ou récupérer) de l'une des deux manières suivantes
  1. En utilisant le bloc *try\catch*
  2. En utilisant le mot clé *throws*

## 6. Le bloc `try\catch` (1)

- La clause ***try*** s'applique à un bloc d'instructions correspondant au fonctionnement normal mais pouvant générer des erreurs.

```
try {  
    // instructions  
}
```



## 6. Le bloc try\catch (2)

- La clause catch s'applique à un bloc d'instructions définissant le traitement d'un type d'erreur. Ce traitement sera lancé sur une instance de la classe d'exception passée en paramètre.

```
try {  
    // instructions  
}  
Catch (ExceptionType1 e1) {  
    // instructions  
}  
Catch (ExceptionType2 e2) {  
    // instructions  
}
```

## 6. Le bloc try\catch(3)

- Exemple

```
public class TestException {  
  
    public static void main(String[] args) {  
  
        int i = Integer.parseInt (args [0]);  
        int j = Integer.parseInt (args [1]);  
        System.out.println("résultat = " + (i / j));  
        System.out.println("la suite du programme");  
    }  
}
```

## 6. Le bloc try\catch (4)

- Exemple

```
public class TestException {  
    public static void main(String[] args) {  
        int i = Integer.parseInt (args [0]);  
        int j = Integer.parseInt (args [1]);  
        try {  
            System.out.println("résultat = " + (i / j));  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Divion par zéro ");  
        }  
        System.out.println("la suite du programme")  
    }  
}
```

Divion par zéro  
la suite du programme

## 6. Le bloc try\catch (5)

- Exemple

```
public class TestException {  
    public static void main(String[] args) {  
        int i = Integer.parseInt (args [0]);  
        int j = Integer.parseInt (args [1]);  
        try {  
            System.out.println("résultat = " + (i / j));  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Divion par zéro " + e.getMessage());  
        }  
        System.out.println("la suite du programme") .  
    }  
}
```

Divion par zéro / **by zero**  
la suite du programme



## 6. Le bloc try\catch (6)

- Exemple

```
public class TestException {  
    public static void main(String[] args) {  
        int i = Integer.parseInt (args [0]);  
        int j = Integer.parseInt (args [1]);  
        try {  
            System.out.println("résultat = " + (i / j));  
        }  
        catch (ArithmeticException e) {  
            e.printStackTrace();  
        }  
        System.out.println("la suite du programme");  
    }  
}
```

java.lang.ArithmeticException: / by zero  
at Test.main(Test.java:8)  
la suite du programme

# ***Que se passe t-il quand une exception est levée ?***

- A tout moment, une instruction ou une méthode peut lever(lancer) une exception
- Une méthode peut attraper (récupérer, saisir) une exception ou bien la laisser se **propager** en la laissant remonter vers la méthode appelante qui peut elle même l'attraper ou la laisser remonter

# ***Que se passe t-il quand une exception est levée ?***

## **Cas 1. En dehors d'un bloc try**

1. La méthode retourne immédiatement, l'exception remonte vers la méthode appelante.
2. La main est donnée à la méthode appelante.
3. L'exception peut alors éventuellement être attrapée et traitée. par cette méthode appelante ou remonter dans la pile.

# ***Que se passe t-il quand une exception est levée ?***

## **Cas 2. Dans un bloc try**

Si une des instructions du bloc try provoque une exception

1. Les instructions suivantes du bloc try ne sont pas exécutées
2. Si au moins une des clauses catch correspond au type de l'exception,
  - la première clause catch appropriée est exécutée,
  - l'exécution se poursuit juste après le bloc try/catch.

Sinon,

- la méthode retourne immédiatement.
- l'exception remonte vers la méthode appelante.



# ***Que se passe t-il quand une exception est levée ?***

## **Remarques (1)**

1. Dans les cas où l'exécution des instructions de la clause try ne provoque pas des exceptions
  - le déroulement du bloc de la clause try se déroule comme s'il n'y avait pas de bloc try-catch
  - le programme se poursuit après le bloc try-catch
2. Si le bloc catch est vide (aucune instruction entre les accolades) alors l'exception capturée est ignorée.

Une telle utilisation de l'instruction try/catch n'est pas une bonne pratique : il est préférable de toujours apporter un traitement adapté lors de la capture d'une exception.

## 5. Traitement d'une exception (10)

*Que se passe t-il quand une exception est levée ?*

### Remarques (2)

3. Eviter d'attraper une exception en mettant dans le bloc catch un simple affichage de message qui ne donne pas d'informations sur le problème. Au minimum afficher ce qu'affiche la méthode `printStackTrace` afin de pouvoir résoudre le problème
4. Si une variable est déclarée à l'intérieur d'un bloc try, elle ne peut pas être utilisée à l'intérieur d'un bloc catch. Il faut déclarer les variables communes à l'extérieur du bloc try/catch

## *Que se passe t-il quand si l'exception n'est pas traitée?*

Si une exception remonte jusqu'à la méthode main sans être traitée par cette méthode (donc pas traitée par les autres méthodes non plus),

- L'exécution du programme est stoppée
- Le message associé à l'exception est affiché, avec une description de la pile des méthodes traversées par l'exception

## 7. Gestion de plusieurs exceptions (1)

- S'il y a plusieurs types d'erreurs et d'exceptions à intercepter, il faut définir autant de bloc catch que de types d'événements.
- Par type d'exception, il faut comprendre « qui est du type de la classe de l'exception ou d'une de ses sous classes ». Ainsi dans l'ordre séquentiel des clauses catch, **un type d'exception ne doit pas venir après un type d'une exception d'une super classe.**

Il faut faire attention à **l'ordre** des clauses **catch** pour traiter en premier les exceptions les plus précises (sous classes) avant les exceptions plus générales.

Un message d'erreur est émis par le compilateur dans le cas contraire.



## 7. Gestion de plusieurs exceptions (2)

```
try {  
    < bloc de code à protéger >  
}  
catch ( TypeException1 E ) { <Traitement TypeException1 > }  
catch ( TypeException2 E ) { <Traitement TypeException2 > }  
.....  
catch ( TypeExceptionk E ) { <Traitement TypeExceptionk > }
```

Où TypeException1, TypeException12, ... , TypeExceptionk sont des classes d'exceptions obligatoirement toutes **distinctes**.

Seule une seule clause **catch** ( TypeException E ) {...}est exécutée (celle qui correspond au bon type de l'objet d'exception instancié).

## 7. Gestion de plusieurs exceptions (3)

```
public class TestException {  
  
    public static void main(String[] args) {  
        // Insert code to start the application here.  
        int i = Integer.parseInt (args [0]);  
        int j = Integer.parseInt (args [1]);  
        try {  
            System.out.println("résultat = " + (i / j));  
        }  
        catch (Exception e) {  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Divion par zéro ");  
        }  
    }  
}
```

**Ne compile pas**  
Bloc catch ne sera  
jamais exécutée, il  
faut donc inverser  
l'ordre de  
déclaration des  
clauses catch.

## 7. Gestion de plusieurs exceptions (4)

Depuis Java 7 : le multi-catch

```
public class TestMultiCatch {  
    public static void main(String[] args) {  
        int A ;  
        try {  
            A = 1 / Integer.parseInt (args [0] ) ;  
        }  
        catch ( ArithmeticException | IndexOutOfBoundsException ex )  
        {  
            System.out.println ( " Index "+ex.getMessage() ) ;  
        }  
        System.out.println ( "la suite du programme..." );  
    }  
}
```

## 8. Le bloc Finally (1)

- Nous avons vu qu'une exception provoque l'arrêt de l'exécution d'une méthode pour se brancher vers le bloc catch adéquat.
- Mais on peut introduire un bloc d'instructions qui seront toujours exécutées (qu'une exception soit levée ou pas).
  - Soit après le bloc try si celui-ci s'est bien déroulé(aucune exception n'est déclenchée)
  - Soit après le bloc catch dans le cas où une exception a été déclenchée



## 8. Le bloc **Finally** (1)

- Ce bloc est introduit par le mot-clé *finally* et **doit obligatoirement être placé après le dernier bloc catch.**

```
try { ..... }  
catch (Ex e) { ..... }  
finally  
{ // instructions A  
}
```

- **Remarques:** Les instructions dans le bloc **finally** sont souvent utilisées pour faire des actions de « nettoyage » (ou de libération de ressources), comme fermer un fichier ou une connexion réseau.

## 9. Lancement d'une exception avec throws (1)

- Toute méthode pouvant lancer une exception checked (contrôlée) doit contenir soit un bloc try/catch (si elle traite l'exception localement) ou bien utiliser le mot clé throws (si l'exception est traitée par une autre méthode).

```
typeDeRetour nomMethode (listeDesArgumentsAvecType)  
    throws a,b,c  
{  
    // instructions  
}
```

**a, b et c sont des  
classes d'exception.**

## 9. Lancement d'une exception avec throws (2)

- Exemple:

```
public class TestThrows {  
    public void test(String[] a) throws ArithmeticException  
    {  
        int i = Integer.parseInt (a[0]);  
        int j = Integer.parseInt (a[1]);  
  
        System.out.println("résultat = " + (i / j));  
    }  
    ...  
}
```

## 9. Lancement d'une exception avec throws (3)

### Le mot clé throw

- Le programmeur peut lancer lui-même des exceptions depuis les méthodes qu'il écrit
  - Il peut lancer des exceptions des classes d'exceptions fournies par le JDK
  - Ou lancer des exceptions appartenant à de nouvelles classes d'exceptions, qu'il a lui-même écrites, et qui sont mieux adaptées aux classes qu'il a écrit.
- Si on lance une exception dans une méthode avec **throw**, il faut l'indiquer dans la déclaration de la méthode, en utilisant le mot clé **throws**.



## 9. Lancement d'une exception avec throws (3)

### exemple

```
public class TestThrow {  
  
    public void test(String[] a) throws  
        ArithmeticException{  
        int i = Integer.parseInt (a [0]);  
        int j = Integer.parseInt (a [1]);  
        if(j==0) throw new ArithmeticException();  
  
        System.out.println("résultat = " + (i / j));  
    }  
}
```

## 10. Les exceptions personnalisées (1)

- En cas de nécessité, on peut créer ses propres exceptions en créant des sous classes de la classe Exception (par convention, le mot « Exception » est inclus dans le nom de la nouvelle classe).
- il suffit ensuite d'utiliser le mot clé throw, suivi d'un objet dont il est instance de la classe créée.

## 10. Les exceptions personnalisées (2)

### Exemple

- Supposons que l'on souhaite manipuler des points ayant des coordonnées non négatives. Nous utiliserons donc une classe Point ayant un constructeur à deux arguments.
- Au sein de ce constructeur, on vérifiera la validité des coordonnées entrées par l'utilisateur, si l'une d'entre elles est incorrecte, nous déclencherons une exception à l'aide du mot clé throw.

## 10. Les exceptions personnalisées (3)

### Exemple

- L'instruction throw a besoin d'avoir un objet du type de l'exception concernée
- Nous créerons donc une classe que nous appellerons CoordonneesException qui dérive de Exception

```
class CoordonneesException extends Exception {  
  
}
```



## 10. Les exceptions personnalisées (4)

```
class Point
{
    private int x;
    private int y;

    public Point(int x, int y) throws
        CoordonneesException
    {
        if ((x<0) || (y<0)) throw new
            CoordonneesException();
        this.x = x;
        this.y = y;
    }
}
```

## 10. Les exceptions personnalisées (5)

```
public class ExceptionPersonnalisee {  
  
    public static void main(String args[]) {  
        int x = Integer.parseInt(args[0]);  
        int y = Integer.parseInt (args[1]);  
  
        try{  
            Point p = new Point (x,y);  
        }  
        catch(CoordonneesException e)  
        {  
            System.out.println(" coordonnée négative ");  
        }  
    }  
}
```

## 10. Les exceptions personnalisées (6)

```
public class ExceptionPersonnalisee {  
  
    public static void main(String args[]) throws  
        CoordonneesException{  
  
        int x = Integer.parseInt(args[0]);  
        int y = Integer.parseInt (args[1]);  
  
        Point p = new Point (x,y);  
    }  
}
```

## 11. Utilisation de throws

### Remarque

- Si une méthode m2 avec un en-tête qui contient « throws XXException » et une méthode m1 fait un appel à m2, alors
  - soit m1 insère l'appel de m2 dans un bloc « try - catch(XXException) »
  - soit m1 déclare (en utilisant le mot clé throws) qu'elle peut lancer une XXException (ou un type plus large)



## 12. La classe Throwable (1)

### Remarque

- C'est la classe mère de toutes les exceptions et elle descend directement de la classe Object.
- Ses principales méthodes sont :
  - **String getMessage( )** retourne le message
  - **void printStackTrace( )** affiche le message d'erreur et la pile des appels de méthodes qui ont conduit au problème
  - **void printStackTrace(PrintStream s)** Idem mais envoie le résultat dans un flux

## 12. La classe Throwable (2)

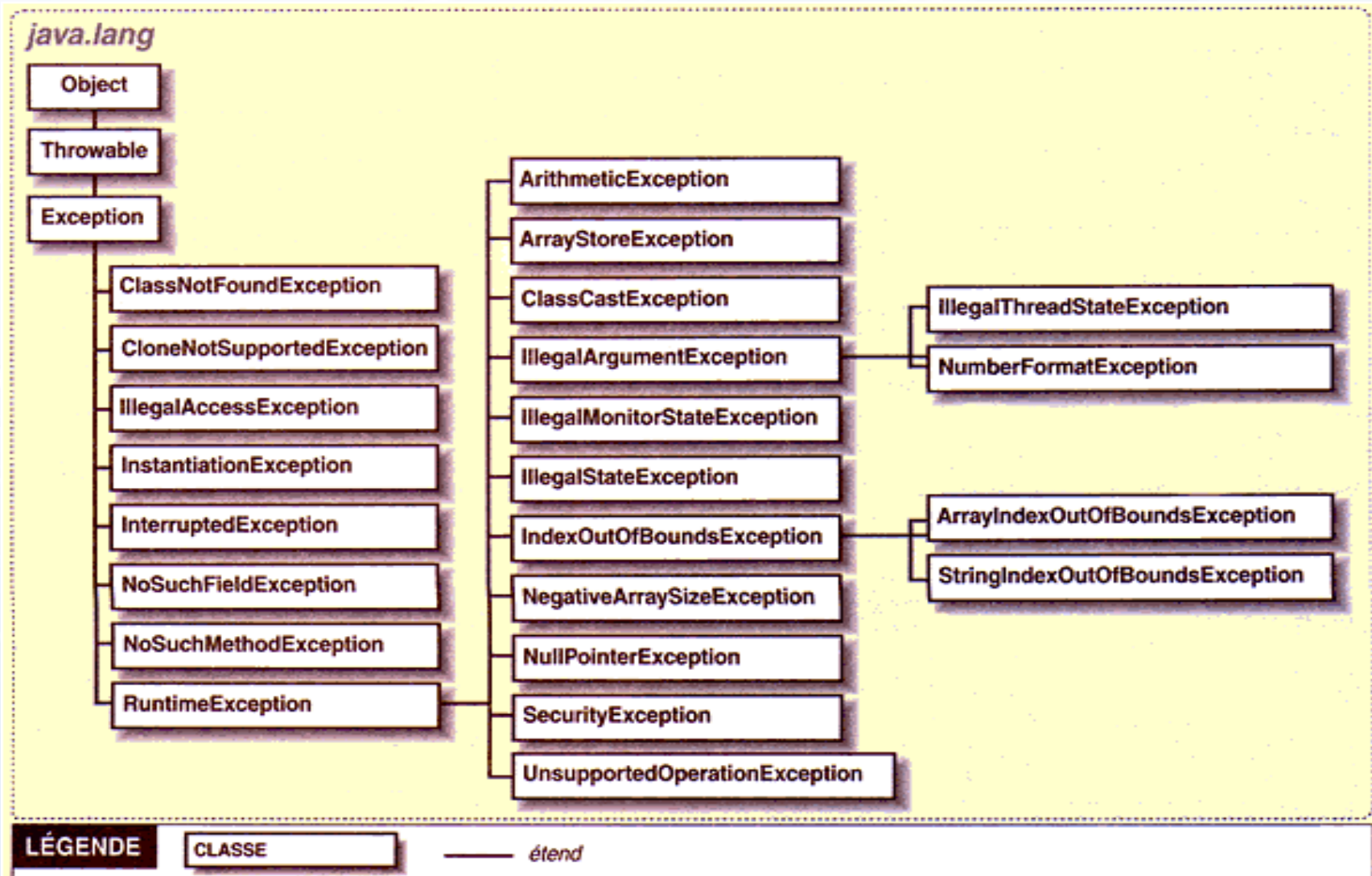
```
public class TestMethodesThrowable {  
    public static void main(java.lang.String[] args) {  
        int i = 3;  
        int j = 0;  
        try {  
            System.out.println("résultat = " + (i / j));  
        }  
        catch (ArithmeticException e) {  
            System.out.println("getmessage");  
            System.out.println(e.getMessage());  
            System.out.println(" ");  
            System.out.println("printStackTrace");  
            e.printStackTrace();  
        }  
    }  
}
```

## 13. Quelques exceptions prédéfinies

Voici quelques exceptions prédéfinies dans Java (sous classe de RuntimeException ):

- `java.lang.NullPointerException` : utilisation de `length` ou accès à une case d'un tableau valant `null` (c'est à dire non encore créé par un `new`).
- `java.lang.ArrayIndexOutOfBoundsException` : accès à une case inexistante dans un tableau.
- `java.lang.NegativeArraySizeException`: générée lorsque quelqu'un souhaite créer un tableau de taille négative.
- `java.lang.IndexOutOfBoundsException`: accès à une case inexistante dans les modèles de donnée indexés (`Vector`, `ArrayList`, etc).
- `java.lang.StringIndexOutOfBoundsException` : accès au *i* eme caractère d'une chaîne de caractères de taille inférieure à *i*.
- `java.lang.NegativeArraySizeException` : création d'un tableau de taille négative.
- `java.lang.ArrayStoreException`: générer pour indiquer qu'une tentative a été effectuée pour stocker le mauvais type d'objet dans un tableau d'objets.
- `java.lang.NumberFormatException` : erreur lors de la conversion d'une chaîne de caractères en nombre.
- `java.lang.ClassCastException`: on tenté de convertir un objet en une sous-classe dont il n'est pas une instance.
- `java.util.InputMismatchException`: se manifeste lorsque un objet `Scanner` récupère une donnée dont le type ne correspond pas avec le type attendu.

# 13. Quelques exceptions prédéfinies





## Exo 1

```
import java.util.*;

class Exo1{
static int[] T = {17, 12, 15, 38, 29, 157, 89, -22, 0, 5};
static int div(int indice, int diviseur){ return T[indice]/diviseur; }
public static void main(String[] args){
int x, y; Scanner sc=new Scanner(System.in);
System.out.println("Entrez l'indice de l'entier à diviser: ");
x = sc.nextInt(); System.out.println ("Entrez le diviseur: ");
y = sc.nextInt(); System.out.println ("Le résultat de la division
    est: ");
System.out.println (div(x,y));}}
```

## Exo 1: solution

```
import java.util.*;
public class Exo1 {
    .....
    boolean stop=false;
    do{
    try{
        System.out.println("Entrez l'indice de l'entier à diviser: ");
        x = sc.nextInt(); System.out.println ("Entrez le diviseur: ");
        y = sc.nextInt(); System.out.println ("Le résultat de la division est: ");
        System.out.println (div(x,y));stop =true;
    }
    catch(InputMismatchException e){System.out.println(e.getMessage());}
    catch(ArrayIndexOutOfBoundsException e){System.out.println(e.getMessage());}
    catch(ArithmeticException e){System.out.println(e.getMessage());}
    }while (!stop);
    }
}
```

## Exo 2

```
import java.util.*;

public class Matrice {
    private int [][] T;
    public Matrice (int [][] tab ){
        T= tab;
    }
    public int sommeNPremier(int n){
        int s=0,cpt=1,i=0, j=0;
        while (cpt<=n){
            s+=T[i][j];cpt++;  j++;
            if (j==T[i].length) {
                j=0;i++;
            }
        }
        return s;
    }
}
```

```
public static void main(String[] args) {

    Scanner sc =new Scanner(System.in);
    int N=Integer.parseInt(args[0]);
    int M=Integer.parseInt(args[1]);
    int Tab[][]=new int[N][M];
    for (int i=0;i<Tab.length;i++){
        for (int j=0;j<Tab[i].length;j++){
            Tab[i][j]=sc.nextInt();
        }
    }
    Matrice mat=new Matrice(Tab);
    int n=Integer.parseInt(args[2]);
    System.out.println(mat.sommeNPremier(n));
}
}
```